

Name/NetID: Nathan Kariuki / nkari011

Session Date: April 30, 2026 (1-2 hours)

Objective: Implement serial BFS using dense worklists and the two parallel BFS implementations. Ensure that all four implementations are consistent.

Attempts made:

The dense worklist adds a frontier bitmap so only active vertices get processed each round. I initially forgot to put the source vertex in the frontier so nothing got processed at all. For the parallel versions the lock pool wraps every write since multiple threads can push to the same destination.

Solutions/Workarounds:

Dense worklist uses two bitmaps that swap each round:

```
void Process(int tid, uint32_t u, const CsrGraph& g) override {
    if (!in_frontier[u]) return;
    for (uint32_t i = g.offsets[u]; i < g.offsets[u+1]; i++) {
        uint32_t v = g.edges[i];
        int candidate = prev[u] + 1;
        if (dist[v] == -1 || candidate < dist[v]) {
            dist[v] = candidate;
            in_next[v] = 1;
            updated[tid] = 1;
        }
    }
}
```

Parallel naive push adds locks around the write:

```
pool.lock(v);
if (dist[v] == -1 || candidate < dist[v]) {
    dist[v] = candidate;
    updated[tid] = 1;
}
pool.unlock(v);
```

Commits: Add BFS serial dense worklist push; Add BFS parallel naive push with striped lock pool; Add BFS parallel dense worklist push with locks

Experiments/Analysis:

Ran all 4 BFS variants on small graphs and full LiveJournal dataset:

dense serial vs naive serial passed
naive parallel vs naive serial passed
dense parallel vs naive serial passed
naive serial vs Phase 2a passed

Learning outcome:

The dense worklist skips vertices that haven't changed so BFS gets faster quickly as most vertices settle after a few rounds. The parallel versions need locks even though the serial ones don't, push style means the destination is no longer owned by one thread.